TECHNICAL DESIGN DOCUMENT

# Theta Chat Bot

## Comprehensive Architecture, API, and AI Guardrails Strategy

| Field | Value |
| --- | --- |
| Project Name | Theta Chat Bot |
| Prepared For | Theta Technolabs |
| Author | Shramika Chavda |
| Document Version | 1.0 |
| Document Status | Internal Use Only |
| Date | July 2026 |

SHEET INDEX

# Contents



echnical

SHEET INDEX

# Contents (Continued)



echnical

# Chapter 1 – Executive Summary

## 1.1 Introduction

The **Theta Chat Bot** is a modular, AI-powered virtual assistant designed to enhance user interaction on the **Theta Technolabs website**. It provides instant, accurate responses based exclusively on the company's publicly available website content, services, portfolios, case studies, careers, and blogs.

The system utilizes a **Retrieval-Augmented Generation (RAG)** architecture. User queries are matched against indexed website content before generating responses through a Large Language Model (LLM). This approach ensures response quality, featuring model-agnostic flexibility to support high-performance or cost-effective AI providers as needed.

To support different business requirements, the proposed solution offers two deployment models:

- **Lightweight Architecture**: Asynchronous API Backend + Vector Database + LLM Engine, optimized for quick deployment and low infrastructure overhead.
- **Enterprise Architecture**: Asynchronous API Backend + Relational & Vector Database + Admin Dashboard, providing advanced analytics, API usage tracking, and operational monitoring.



The chatbot is embedded into the website as a **floating chat widget** using a lightweight frontend component integrated with the current website platform. The entire application is fully **containerized**, allowing seamless deployment on an on-premise company server or cloud infrastructure.

## 1.2 Why This Solution?

Modern websites contain extensive information spread across multiple pages, making it time-consuming for visitors to locate specific details. Users expect conversational interfaces that interpret natural language and deliver concise, context-aware answers.

The proposed chatbot transforms the website into an intelligent knowledge assistant. Instead of relying on manually curated FAQs or hardcoded decision rules, the system dynamically retrieves relevant information directly from indexed website content and uses an LLM to generate human-like responses grounded in that verified context.

This approach offers several key business advantages:

- **Reduces Support Workload**: Minimizes repetitive customer support and sales inquiries.
- **Enhances User Engagement**: Guides website visitors through conversational discovery.
- **Continuous Alignment**: Ensures responses automatically reflect website updates without retraining.
- **Scalable Foundation**: Provides a modular architecture for dashboards and analytics.

## 1.3 Project Goals



The primary business objectives of the chatbot are:

- **Improve Visitor Experience**: Deliver immediate, 24/7 conversational assistance on website offerings.
- **Reduce Support Workload**: Automate responses to common inquiries for sales and support teams.
- **Increase Engagement & Conversion**: Guide users to relevant service, portfolio, and contact pages.
- **Ensure High Accuracy**: Provide reliable answers based strictly on approved company content.
- **Provide Scalable AI Platform**: Establish an extensible architecture that grows with business needs.

## 1.4 Key Features

The chatbot delivers the following technical and functional capabilities:

- **Semantic Search**: Understands user intent and natural language queries beyond exact keyword matching.
- **Context-Aware Answering**: Dynamically retrieves relevant website chunks before response generation.
- **Source Citations & Links**: Includes direct page references for transparent verification.
- **Session-Based Conversation Memory**: Maintains context across follow-up questions within active sessions.
- **Out-of-Scope & Fallback Handling**: Politely declines off-topic queries and directs users to contact channels.
- **Embeddable UI Widget**: Floating chat interface accessible across all website pages or header navigation.

## 1.5 Scope

| In Scope (Supported Queries) | Out of Scope (Unsupported) |
|---|---|
| • Company information • AI Development • Mobile App Development • Web Development • IoT Solutions • Blockchain Solutions • Cloud Services • UI/UX Design • Case Studies • Blogs • Careers • Contact Information • Company Technologies • FAQs | • Personal opinions • Medical advice • Legal advice • Financial advice • Current news • Internet search results • Information unrelated to Theta Technolabs |

## 1.6 High-Level Solution

The chatbot operates on a **Retrieval-Augmented Generation (RAG)** pipeline. Instead of storing hardcoded answers or relying on pre-trained model memory, the system executes a four-step processing flow for every user query:

1. **User Query Processing** Accepts and validates user natural language input.
2. **Semantic Knowledge Retrieval** Searches the vector database to extract the most relevant website chunks.
3. **Contextual Prompt Construction** Injects retrieved chunks and system guardrails into a structured prompt.
4. **Grounded Response Delivery** Generates a concise, verified answer with source references.

## 1.7 Why This Architecture?

The technical architecture is selected based on key operational advantages:

- **Why RAG over Fine-Tuning**: Eliminates expensive AI model retraining whenever website content changes.
- **Why Vector Search Database**: Delivers sub-second semantic retrieval across unstructured company content.
- **Why Asynchronous API Layer**: Ensures high concurrency, low latency, and efficient request handling.
- **Why Containerized Deployment**: Guarantees environment consistency, portability, and on-premise security.
- **Why Modular Component Design**: Permits independent upgrades of LLMs, vector stores, or UI widgets.

## 1.8 Expected User Experience

A visitor opens the **Theta Technolabs website** and notices the **chat icon** in the bottom-right corner (or optionally in the website header).

The visitor asks:

> **VISITOR QUESTION**
> "Do you develop **AI Agents**?"

The chatbot searches the knowledge base, retrieves the relevant AI services content, generates a concise answer, and responds with:

> **CHATBOT RESPONSE**
> "Yes. Theta Technolabs develops custom **AI agents**, **conversational AI** solutions, **LLM** applications, and **intelligent automation** systems tailored to business needs."

It then provides:

> **RELATED LINK**
> Related Page: https://www.thetatechnolabs.com/artificial-intelligence



This allows users to both receive an immediate answer and continue exploring the official website.

# Chapter 2 – Solution Overview

## 2.1 Solution Overview

The Theta Chat Bot is designed as an AI-powered Retrieval-Augmented Generation (RAG) system that delivers accurate responses using information exclusively retrieved from the Theta Technolabs website.

Rather than storing predefined question-answer pairs or training a custom language model, the chatbot dynamically retrieves the most relevant website content and uses it as contextual input for the Large Language Model (LLM). This enables the chatbot to answer user queries naturally while remaining grounded in verified company information.

The chatbot follows a modular architecture consisting of five primary stages:

- **Knowledge Acquisition** – Website content is collected and prepared for indexing.
- **Knowledge Storage** – Processed content is stored in a searchable vector database.
- **Query Processing** – User questions are analyzed and matched against the knowledge base.
- **Response Generation** – Retrieved context is combined with the user query and sent to the LLM to generate a final response.
- **Response Delivery** – The generated answer is returned to the user through the website chat interface.

PIPELINES

This architecture separates knowledge storage from language generation, allowing website content to be updated independently without retraining the AI model.

## 2.2 Why Retrieval-Augmented Generation (RAG)?

Several architectural approaches were evaluated before selecting the final solution. The following table compares the available options.

| Approach | Description | Advantages | Limitations |
|---|---|---|---|
| Rule-Based Chatbot | Predefined intents and responses | Simple, predictable | Difficult to maintain, poor scalability, limited conversational ability |
| FAQ Matching | Keyword-based search through predefined FAQs | Easy implementation | Cannot answer complex or unseen questions |
| Fine-Tuned LLM | Train a custom model using company data | High customization | Expensive, time-consuming, requires retraining whenever content changes |
| Retrieval-Augmented Generation (Selected) | Retrieves relevant website content and provides it to the LLM before generating a response | Accurate, scalable, easier to maintain, supports dynamic website updates | Requires vector database and retrieval pipeline |



ARCHITECTURE DECISION — WHY RAG?

**1. Single Source of Truth**
The Theta website is the primary source of truth and will evolve over time as new services, case studies, blogs, or technologies are added.

**2. Reduced Operational Overhead**
Fine-tuning a language model whenever website content changes would introduce unnecessary operational overhead and increased costs.

**3. Seamless Updates**
RAG eliminates this challenge by separating knowledge management from language generation. Website content can be re-indexed whenever updates occur, allowing the chatbot to access the latest information without retraining the underlying LLM.

**4. Accuracy and Flexibility**
This approach grounds responses in retrieved website content, improves maintainability, and provides flexibility to switch between different LLM providers.

## 2.3 System Workflow

flowchart TB %% ========== %% Nodes %% ========== A([User Query]) subgraph Query_Processing B[Query Processing] C[Embedding Generation] D[Embedding Model] end subgraph Knowledge_Retrieval E[Semantic Search] F[(Vector Database)] G[Ranking Engine] H[Context Builder] I[Metadata Retrieval] end subgraph Response_Generation J[Prompt Builder] K[LLM Provider] L[Response Formatter] M[Attach Source References] end N([Final Response]) %% ========== %% Flow %% ========== A --> B B --> C C --> D D --> E E --> F F --> G G --> H H --> I I --> J J --> K K --> L L --> M M --> N %% ========== %% Colors %% ========== style A fill:#0d2b45,stroke:#e8a33d,stroke-width:3px,color:#ffffff style N fill:#0d2b45,stroke:#e8a33d,stroke-width:3px,color:#ffffff style B fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style C fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style D fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style E fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style G fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style H fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style I fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style J fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style K fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style L fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style M fill:#1c5d8c,stroke:#0d2b45,color:#ffffff style F fill:#ffffff,stroke:#e8a33d,stroke-width:3px,color:#0d2b45

**Figure 2.1: System Workflow**

The workflow is divided into three core phases:

- **Query Processing:** Converts the user's natural language input into mathematical embeddings.
- **Knowledge Retrieval:** Searches the vector database for the most relevant website content based on semantic similarity.
- **Response Generation:** Combines the retrieved knowledge with the user query to construct an accurate, grounded response via the LLM.

## 2.4 Key Design Decisions

| Decision | Rationale |
|---|---|
| Website as the primary knowledge base | Ensures responses remain aligned with official company information. |
| RAG instead of fine-tuning | Simplifies updates and reduces operational costs. |
| FastAPI backend | High performance, asynchronous request handling, and excellent AI ecosystem support. |
| Gemini as the initial LLM | Cost-effective with strong reasoning capabilities for MVP deployment. |
| Modular architecture | Allows future replacement of the LLM, vector database, or frontend without redesigning the system. |
| Docker deployment | Simplifies deployment, portability, and on-premise hosting. |
| Optional PostgreSQL dashboard | Enables analytics and operational monitoring without affecting the core chatbot functionality. |

# Chapter 3 – Knowledge Management Pipeline

## 3.1 Overview

The success of the Theta Chat Bot depends on the quality, accuracy, and freshness of its knowledge source. To ensure reliable and consistent responses, the chatbot uses the official website data exclusively. Every response generated by the chatbot is based on publicly available information published on the site.

Website content is transformed into a structured semantic knowledge base that can be efficiently searched during every user interaction.

The Knowledge Management Pipeline consists of the following stages:

- Content Acquisition
- Content Extraction & Cleaning
- Content Chunking
- Embedding Generation
- Knowledge Storage
- Knowledge Synchronization

KNOWLEDGE

Each stage is independent and modular, allowing future improvements without affecting the rest of the system.

## 3.2 Content Acquisition

The first stage is responsible for collecting publicly available content from the Theta Technolabs website. The crawler indexes only pages that provide meaningful business information.

| Content Category | Included |
|---|---|
| Home Page | ✓ |
| About Us | ✓ |
| Services | ✓ |
| Technologies | ✓ |
| Industry Solutions | ✓ |
| Portfolio / Case Studies | ✓ |
| Blogs | ✓ |
| Careers | ✓ |
| Contact Information , FAQ | ✓ |

KNOWLEDGE

Pages such as privacy policies, cookie notices, legal disclaimers, or other pages that do not contribute to answering user queries may be excluded from indexing.

## 3.3 Content Extraction & Cleaning

Before any information is stored within the knowledge base, the website content undergoes a preprocessing pipeline to ensure that only meaningful and relevant information is indexed.

The extraction and cleaning process includes:

- HTML parsing and structured text extraction
- Removal of navigation menus
- Removal of headers and footers
- Removal of cookie banners and policy notices
- Removal of JavaScript and CSS elements
- Elimination of duplicate content
- Normalization of whitespace and formatting
- Preservation of headings and document hierarchy
- Metadata extraction (Page Title, URL, Section Name)
- Validation to discard empty or low-quality content

Only clean, structured, and meaningful textual content proceeds to the embedding stage, ensuring high-quality semantic retrieval and reducing irrelevant search results.

## 3.4 Content Chunking

Since modern language models and vector databases perform better with smaller, focused contextual units, each webpage is divided into multiple semantic chunks before embeddings are generated.

The system utilizes a **Sliding Window Overlap Chunking Strategy**. A **500-token chunk size** (approx. 350–400 words) with a **50–100 token overlap** is used as a baseline example in this document. Note that the exact chunk size and character/token boundaries are fully configurable and will be adjusted dynamically at runtime based on the document type, content density, and model context window optimization.

The chunking strategy follows these core principles:

- **Configurable Chunk Size**: 500-token baseline (adjustable at runtime) optimized for embedding models and vector retrieval.
- **Overlapping Window**: 50–100 token overlap prevents context loss across chunk boundaries.
- **Paragraph & Sentence Awareness**: Preserves logical paragraph and sentence boundaries whenever possible.
- **Heading Context Preservation**: Keeps headings attached to their respective body text chunks.
- **Metadata Attachment**: Stores URL, page title, section name, and chunk index with every vector record.

> CHUNKING STRATEGY JUSTIFICATION
>
> **1. Context Cohesion (500 Tokens):** Smaller chunks (e.g., 100-200 tokens) often lose the broader context, forcing the LLM to guess connections. 500 tokens provide enough surrounding text to fully answer multi-part questions while remaining granular enough for accurate vector search.
>
> **2. Edge-Case Prevention (Overlap):** The 50–100 token overlapping window prevents critical information from being split across two separate chunks, ensuring the retrieval engine doesn't miss a fact that spans a boundary.

Example (Baseline Configuration):

| Chunk ID | Page | Section | Description |
|---|---|---|---|
| CH-001 | AI Development Services | Artificial Intelligence | Service Overview (~500 tokens example) |
| CH-002 | AI Development Services | Machine Learning | Service Details (~500 tokens, 50-token overlap) |
| CH-003 | Portfolio | Healthcare Solution | Project Description (~500 tokens example) |

This improves retrieval precision by allowing the chatbot to retrieve only the most relevant section instead of an entire webpage.

## 3.5 Embedding Generation

After chunking, each content block is converted into a numerical vector representation known as an embedding.

Embeddings capture the semantic meaning of text rather than relying solely on keyword matching. This enables the chatbot to understand user intent even when different words or phrases are used.

For example:

- **User Question:** "Do you build AI chatbots?"

- **Website Content:** "Theta Technolabs develops intelligent conversational AI solutions."

Although the wording differs, the embedding model recognizes the semantic similarity and retrieves the correct content.

## 3.6 Knowledge Storage

The chatbot requires a storage solution that supports semantic search while remaining scalable for future business requirements. The recommended storage architecture depends on whether the application requires only vector search or both vector search and relational data management.

> OPTION 1 — DEDICATED VECTOR DATABASE (QDRANT)
>
> **Option 1 – Dedicated Vector Database (Qdrant).** When the chatbot only requires semantic search over the knowledge base and does not need conversation logging, analytics, or an administrative dashboard, **Qdrant** is the recommended solution. Qdrant is purpose-built for vector similarity search and provides excellent retrieval performance with minimal infrastructure overhead.

### Advantages

`LIGHTWEIGHT DEPLOYMENT` `LOW RESOURCE CONSUMPTION` `HIGH-PERFORMANCE SEARCH` `SIMPLE ARCHITECTURE` `EASY TO DEPLOY & MAINTAIN`

### Limitations

`NO RELATIONAL STORAGE` `EXTRA DB NEEDED FOR LOGS & ANALYTICS`

> OPTION 2 — POSTGRESQL WITH PGVECTOR
>
> **Option 2 – PostgreSQL with pgvector.** When the application requires **conversation history, user feedback, operational metrics, API logs, or an administrative dashboard**, **PostgreSQL with the pgvector extension** is the recommended architecture. In this approach, PostgreSQL stores both relational application data and vector embeddings using the pgvector extension within a single database system, eliminating the need for a separate vector database.

### Advantages

`SINGLE DATABASE SYSTEM` `RELATIONAL & VECTOR UNIFIED` `ANALYTICS & REPORTING` `SIMPLIFIED MAINTENANCE` `ENTERPRISE DASHBOARDS`

### Trade-offs

`HIGHER RESOURCE DEMANDS` `LARGE-SCALE VECTOR LIMITS` `POSTGRESQL ADMIN OVERHEAD`

### Architectural Recommendation

| Requirement Scope | Recommended Architecture | Architectural Rationale & Key Advantages |
|---|---|---|
| Knowledge Retrieval Only | Qdrant (Dedicated Vector DB) | Preferred choice for knowledge retrieval alone. Lightweight and purpose-built/optimized for high-speed semantic search with minimal infrastructure overhead. |
| Retrieval + Conversation Logging, Analytics, User Management or Admin Dashboard | PostgreSQL + pgvector (Unified DB) | Recommended when relational data or dashboards are needed. PostgreSQL manages both relational data and vector embeddings within a single system, making a separate vector database like Qdrant unnecessary and resulting in a simpler, more maintainable architecture. |

KNOWLEDGE

## 3.7 Knowledge Synchronization

The chatbot's knowledge base must remain synchronized with the Theta Technolabs website while avoiding unnecessary processing of unchanged content.

Since website content is updated only when new pages, blogs, services, or portfolio items are published, running scheduled synchronization jobs would repeatedly process unchanged data and increase infrastructure overhead.

To address this, the proposed solution adopts an Event-Driven Incremental Synchronization approach. Whenever website content is published, updated, or removed, the synchronization pipeline is triggered automatically. Instead of rebuilding the entire knowledge base, the system identifies only the affected pages and updates those records.

**Synchronization Approaches Considered**

| Strategy | Description | Advantages | Limitations |
|---|---|---|---|
| Manual Refresh | Administrator manually re-indexes the entire website. | Simple implementation | Time-consuming and dependent on manual intervention. |
| Scheduled Synchronization | Automatically crawls the website at predefined intervals. | Fully automated | Processes unchanged content and increases unnecessary compute costs. |
| Event-Driven Incremental Synchronization (Recommended) | Triggered whenever website content is created, modified, or deleted. Only affected pages are reprocessed. | Efficient, scalable, faster updates, and reduced infrastructure cost. | Requires integration with the website publishing workflow or CMS. |

**Synchronization Workflow**

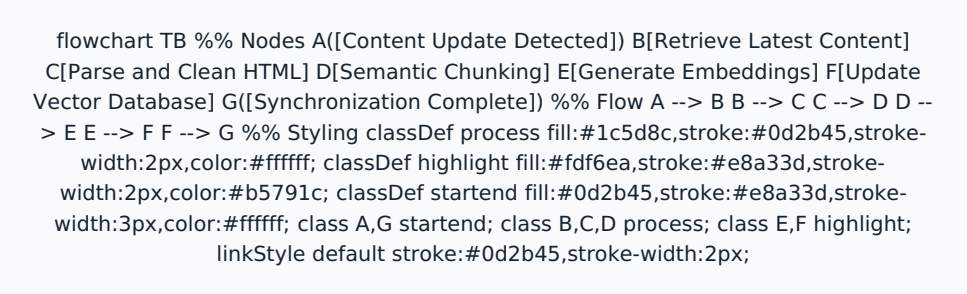


**Figure 3.1: Synchronization Workflow**

Whenever website content changes, the pipeline detects the newly modified pages, cleans the HTML, divides it into semantic chunks, generates fresh embeddings, and replaces only the updated vectors in the database. All other website pages remain untouched.

This approach significantly reduces processing time, minimizes embedding generation costs, and ensures the chatbot always retrieves the most recent website information without reprocessing the complete knowledge base.

## 3.8 Metadata Management

Every indexed content chunk stores metadata to improve retrieval quality, enable source attribution, and support future analytics.

| Metadata Field | Description |
|---|---|
| Chunk ID | Unique identifier for each semantic chunk |
| Page URL | Original source page |
| Page Title | Human-readable page name |
| Section Heading | Contextual heading within the page |
| Content Category | Services, Blogs, Careers, Portfolio, etc. |
| Last Indexed Date | Timestamp of the latest indexing operation |
| Version | Knowledge version identifier |

| | |
|---|---|
| Vector ID | Reference to the stored embedding |

Metadata enables future capabilities such as source citations, document versioning, analytics, debugging, and knowledge auditing.

## 3.9 Knowledge Lifecycle

The complete knowledge processing lifecycle converts raw website pages into search-ready vector embeddings through the following structured flow:

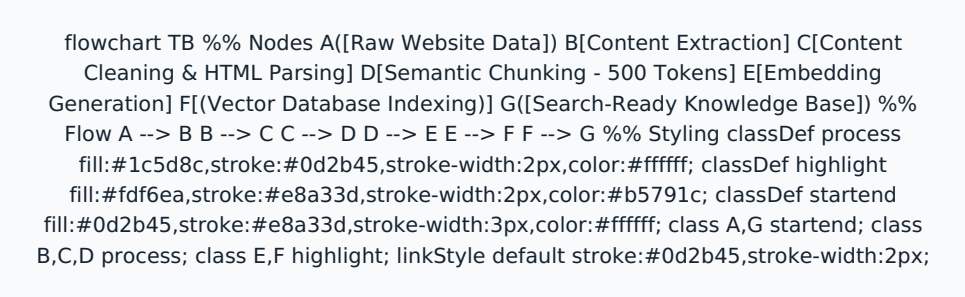


**Figure 3.2: Knowledge Lifecycle**

## 3.10 Key Design Decisions

| Decision | Justification |
|---|---|
| Official website as primary domain | Ensures responses remain accurate and aligned with company information. |
| Comprehensive content cleaning | Removes irrelevant website elements before indexing. |
| Semantic chunking | Improves retrieval precision and contextual relevance. |
| Event-Driven Incremental Synchronization | Updates only modified content, reducing processing overhead. |
| Qdrant for lightweight deployments | Optimized for fast semantic search with minimal infrastructure. |
| PostgreSQL + pgvector for enterprise deployments | Provides semantic search alongside analytics and dashboard capabilities within a single database. |
| Modular pipeline architecture | Enables future enhancements without requiring major architectural changes. |

# Chapter 4 – Retrieval & Response Generation

## 4.1 Overview

The Retrieval & Response Generation Pipeline is the core processing layer of the Theta Chat Bot. It is responsible for transforming a user's natural language query into an accurate, context-aware response by retrieving relevant website content and providing it to the Large Language Model (LLM).

Instead of relying solely on the language model's pre-trained knowledge, the system first searches the organization's indexed knowledge base. Only the retrieved content is supplied to the LLM, ensuring that responses remain grounded in official Theta Technolabs information.

The complete processing pipeline consists of the following stages:

- User Query Reception
- Query Validation
- Query Embedding Generation
- Semantic Retrieval
- Context Construction
- Prompt Engineering
- Response Generation
- Response Validation
- Response Delivery

sequenceDiagram autonumber actor User participant CW as Chat Widget participant API as REST API participant VS as Validation Service participant ES as Embedding Service participant DB as Vector Database participant CB as Context Builder participant PB as Prompt Builder participant LLM as LLM Provider participant RF as Response Formatter User->>CW: Enter Question CW->>API: POST /chat API->>VS: Validate Request VS-->>API: Valid API->>ES: Generate Embedding ES-->>API: Query Vector API->>DB: Semantic Search DB-->>API: Top Matching Chunks API->>CB: Build Context CB-->>API: Context API->>PB: Create Prompt PB-->>API: Final Prompt API->>LLM: Generate Response LLM-->>API: AI Response API->>RF: Format Response RF-->>API: Final JSON API-->>CW: Chat Response CW-->>User: Display Response

**Figure 4.1: Request Processing Sequence**

## 4.2 User Query Reception

A visitor submits a question through the chatbot interface embedded within the Theta Technolabs website.

Example: "*What AI Development services does Theta Technolabs provide?*"

The frontend sends the request securely to the FastAPI backend through a REST API. At this stage the backend performs basic validation including:

- Empty message validation
- Max length validation
- Unsupported characters
- Security filtering
- Rate limiting

Only valid requests proceed to the retrieval pipeline.

## 4.3 Query Embedding Generation

After validation, the user's question is converted into a semantic vector representation using the selected embedding model.

Unlike traditional keyword matching, embeddings capture the meaning and intent of the user's question.

> SEMANTIC MATCHING EXAMPLE
>
> **User Question:** "Can you build AI chatbots?"
>
> **Website Content:** "We provide Conversational AI Development Services."
>
> Although different wording is used, both sentences represent the same intent. The embedding model allows the system to recognize this similarity.

## 4.4 Semantic Retrieval

The generated query embedding is compared against all indexed website embeddings stored in the vector database. The retrieval engine calculates similarity scores between the user's question and every indexed content chunk. The Top-K most relevant chunks are selected.

**Typical Configuration:**

| Parameter | Value |
|---|---|
| Retrieval Method | Semantic Search |
| Similarity Metric | Cosine Similarity |
| Retrieved Chunks (Top-K) | Top 3 to 5 chunks |
| Minimum Similarity Threshold | 0.60 (Strict cutoff) |

> RETRIEVAL PARAMETERS JUSTIFICATION
>
> **1. Top-K Selection (3 to 5):** Retrieving more than 5 chunks increases the context window size, which drives up API costs and can dilute the LLM's focus (the "lost in the middle" phenomenon). Top-K=5 ensures maximum relevant context without overwhelming the prompt.
>
> **2. Similarity Threshold (0.60):** Queries matching below 0.60 are mathematically distant from the official knowledge base. This strict cutoff acts as the first line of defense against out-of-scope questions or hallucination risks.

Only highly relevant chunks are forwarded to the language model.

## 4.5 Context Construction

The retrieved content alone is not sufficient for producing high-quality responses. The backend constructs a structured context containing:

- Retrieved website content
- Source page information
- User question
- Session conversation history (if enabled)
- System instructions

This structured context ensures the language model receives only relevant information.

## 4.6 Prompt Engineering

A carefully designed system prompt controls the behavior of the language model. The prompt instructs the LLM to:

- Answer only using the provided website content.
- Avoid generating unsupported information.
- Clearly state when sufficient information is unavailable.

- Maintain a professional and friendly tone.
- Keep responses concise and relevant.
- Avoid speculation or assumptions.
- Prefer official terminology used by Theta Technolabs.

> **PROMPT STRUCTURE**
>
> **1.** System Instructions
> **2.** Retrieved Context
> **3.** Conversation Context (Optional)
> **4.** User Question

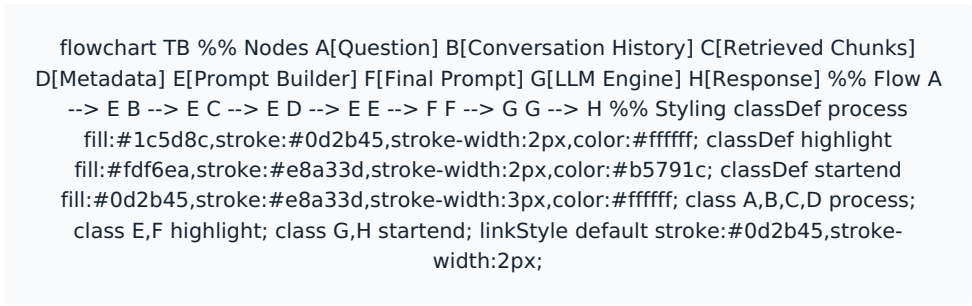


**Figure 4.2: Prompt Assembly Flow**

## 4.7 Response Generation

The constructed prompt is submitted to the configured Large Language Model (LLM Engine). The architecture remains provider-independent, allowing seamless deployment with high-performance or cost-effective AI model providers without changing the backend logic.

The LLM Engine generates a response using only the retrieved context supplied by the backend, ensuring grounded, accurate, and hallucination-free answers.

## 4.8 Conversation Memory

Two conversation modes are considered.

> **OPTION A – STATELESS CONVERSATION**
>
> Each user query is processed independently.

**Advantages**

- Simple implementation
- Lower token consumption
- Better scalability
- No conversation storage

**Limitations**

- Cannot understand follow-up questions

> **OPTION B – SESSION-BASED CONVERSATION**
>
> The chatbot temporarily remembers previous messages within the current browser session.

**Advantages**

- Better conversational experience
- Supports follow-up questions
- Improved contextual understanding

**Limitations**

- Slightly higher token usage
- Requires temporary session management

**Recommended Approach**

The proposed implementation recommends **Session-Based Conversation Memory** for

the active browser session only. Conversation history is discarded once the session expires, ensuring a balance between user experience and resource efficiency.

## 4.9 Response Validation

Before returning the generated response to the user, the backend performs a final validation. The response is evaluated based on:

- Retrieval confidence
- Context relevance
- Response completeness
- Supported knowledge domain

Depending on the confidence level, different actions are taken. *(Note: The specific scoring thresholds underlying these confidence levels are detailed in Section 9.5 – Confidence Guardrail).*

| Confidence Level | System Behavior |
|---|---|
| High | Return the answer normally. |
| Medium | Return the answer with the related website page reference. |
| Low | Inform the user that sufficient information could not be found and recommend contacting Theta Technolabs. |
| Out of Scope | Politely explain that the chatbot only supports Theta Technolabs-related queries and guide the user to the Contact or Inquiry page. |

This strategy improves reliability while preventing misleading responses.

## 4.10 Response Delivery

Once validated, the response is returned to the frontend in JSON format. The chatbot interface displays:

- AI-generated answer
- Optional source page reference
- Helpful / Not Helpful feedback buttons (future enhancement)
- Contact or Inquiry action when required

This provides users with a consistent and intuitive conversational experience.

## 4.11 Complete Processing Flow

The complete end-to-end request handling workflow illustrates how a user's question travels from the frontend floating widget through validation, semantic retrieval, prompt assembly, LLM inference, and response delivery:

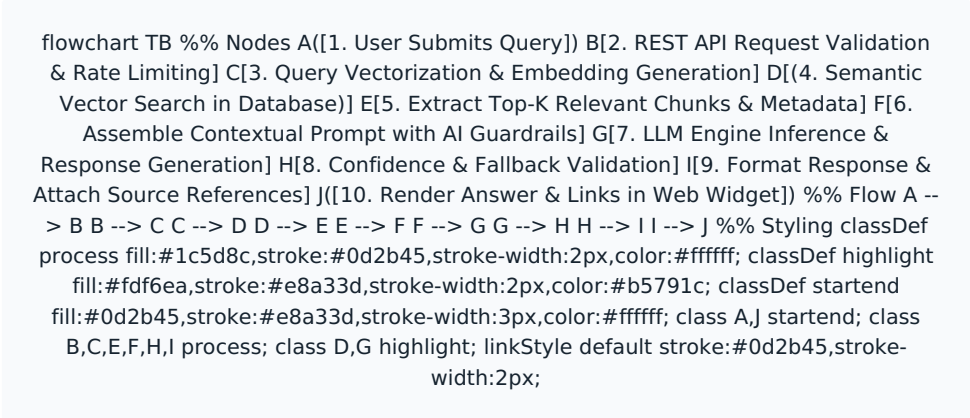


**Figure 4.3: End-to-End Query Handling**



## 4.12 Key Design Decisions

| Decision | Justification |
|---|---|
| Semantic search over keyword search | Better understanding of natural language queries. |
| Prompt engineering | Ensures consistent and grounded responses. |
| Session-based memory | Supports natural conversations without permanent storage. |
| Confidence-based validation | Prevents unsupported or misleading answers. |
| Provider-independent LLM layer | Enables future migration between Gemini and OpenAI with minimal effort. |
| Source-aware responses | Improves transparency and user trust. |

# Chapter 5 – System Architecture

## 5.1 Architecture Overview

The Theta Chat Bot follows a modular, service-oriented architecture designed around the principles of scalability, maintainability, and loose coupling. Each major component has a clearly defined responsibility, allowing individual services to be upgraded or replaced without impacting the rest of the system.

The architecture separates the user interface, application logic, artificial intelligence layer, knowledge storage, and operational services into independent layers. This design ensures the chatbot remains easy to maintain while supporting future enhancements such as analytics dashboards, multilingual support, CRM integration, or migration to different Large Language Models (LLMs).

The overall architecture consists of the following layers:

- Presentation Layer
- API Layer
- Business Logic Layer
- AI & Retrieval Layer
- Data Layer
- Infrastructure Layer

## 5.2 Presentation Layer

The Presentation Layer is the user-facing interface through which visitors interact with the chatbot.

The existing Theta Technolabs website is developed using a no-code website builder (such as Webflow). Instead of rebuilding the frontend, a lightweight custom JavaScript chatbot widget will be embedded into the website.

**Responsibilities**

- Display chat interface
- Capture user questions
- Display AI responses
- Show typing indicators
- Display source references (optional)
- Display fallback actions
- Manage session identifiers
- Communicate securely with backend APIs

Since the chatbot is embedded into the existing website, no additional frontend framework is required.

## 5.3 API Layer

The API Layer acts as the communication bridge between the frontend and backend services.

All requests from the chatbot widget are transmitted through secure REST APIs exposed by the FastAPI backend.

**Responsibilities**

- Receive user messages
- Validate requests
- Generate session identifiers
- Route requests to the AI pipeline

- Return structured JSON responses

**Example API Flow**

Website → Chat Widget → REST API → FastAPI

## 5.4 Business Logic Layer

The Business Logic Layer coordinates the complete chatbot workflow.

It acts as the orchestration layer between the frontend, retrieval engine, vector database, and language model.

**Responsibilities**

- Query validation
- Session management
- Prompt construction
- Context management
- Response validation
- Error handling
- Logging
- Analytics integration (optional)

This layer ensures that all chatbot requests follow a consistent processing pipeline.

## 5.5 AI & Retrieval Layer

The AI Layer is responsible for transforming natural language questions into accurate responses.

Rather than sending user questions directly to the language model, the system first retrieves the most relevant website content using semantic search.

The AI pipeline consists of:

1. Embedding Generation
2. Semantic Retrieval
3. Context Construction
4. Prompt Engineering
5. LLM Response Generation
6. Confidence Validation

This Retrieval-Augmented Generation (RAG) approach significantly improves response accuracy while reducing hallucinations.



## 5.6 Large Language Model (LLM)

The architecture is designed to remain independent of any specific AI provider.

**Primary Provider: Google Gemini**

Reason:

- Lower operational cost
- Strong reasoning capability
- Large context window
- Suitable for MVP deployment

**Future Option: OpenAI**

If future testing indicates that higher reasoning quality or advanced features are required, the LLM layer can be switched to OpenAI without requiring architectural changes.

The abstraction layer ensures that the backend communicates through a common interface regardless of the selected provider.

LLM ABSTRACTION INTERFACE (PSEUDOCODE)

class BaseLLMProvider(ABC): @abstractmethod async def generate_response(self, prompt: str, context: list) -> str: """Generates a grounded response using the provided context.""" pass

## 5.7 Data Layer

The Data Layer stores the knowledge base and, optionally, operational analytics.

Two deployment architectures are proposed.

ARCHITECTURE A – LIGHTWEIGHT DEPLOYMENT

**Components:** Qdrant Vector Database

**Purpose:** Store semantic embeddings, Perform similarity search, Return relevant website content

Recommended when only chatbot functionality is required.

ARCHITECTURE B – ENTERPRISE DEPLOYMENT

**Components:** PostgreSQL + pgvector

**Purpose:** Store semantic embeddings, Store chatbot analytics

**Analytics Data:**

Conversation logs User feedback API usage Token consumption Dashboard metrics Configuration data
Recommended when an Admin Dashboard and operational reporting are required.

## 5.8 Infrastructure Layer

The entire solution will be containerized using Docker to ensure consistent deployment across development, testing, and production environments.

Each major service runs inside an isolated container.

**Possible containers include:**

- FastAPI Application
- Nginx Reverse Proxy
- Qdrant (Lightweight Option)
- PostgreSQL + pgvector (Enterprise Option)
- pgAdmin (Enterprise Option)

**Containerization provides:**

- Simplified deployment
- Easy maintenance
- Environment consistency
- Portability
- On-Premise compatibility

## 5.9 Deployment Strategy

The proposed deployment model is an on-premise Docker environment hosted within the company's infrastructure.

**Advantages include:**

- Complete control over infrastructure
- Internal data management
- Easier compliance with company policies
- Lower long-term operational costs
- Simplified backup and maintenance

If future scalability requirements increase, the same Docker architecture can be deployed on cloud platforms with minimal modifications.

## 5.10 Component Communication


**Figure 5.1: Component Communication**

The system components communicate in a strictly sequential flow: the frontend passes user queries to the FastAPI backend, which retrieves semantically similar vectors, constructs a prompt, invokes the LLM, validates the response, and returns the final answer along with source citations.

## 5.11 Scalability Considerations

The proposed architecture supports future enhancements without requiring major redesign.



**Possible future extensions include:**

- Admin Dashboard
- CRM Integration
- Multilingual Support
- Voice-based Chatbot
- Multi-Website Knowledge Base
- Document Upload Support
- Hybrid Search (Semantic + Keyword)
- Response Caching
- Redis Integration
- Multiple LLM Providers

This ensures that new capabilities can be introduced independently while maintaining compatibility with the existing system.

## 5.12 Key Design Decisions

| Decision | Justification |
|---|---|

| | |
|---|---|
| Modular layered architecture | Simplifies maintenance and future enhancements. |
| Existing website integration | Eliminates the need to rebuild the frontend. |
| FastAPI as the orchestration layer | Provides high-performance asynchronous API handling. |
| RAG architecture | Grounds responses in official website content. |
| Docker-based deployment | Ensures portability and deployment consistency. |
| On-premise deployment | Keeps infrastructure under company control while reducing long-term operational costs. |
| Provider-independent AI layer | Enables migration between Gemini and OpenAI without changing the overall architecture. |
| Dual database strategy | Supports both lightweight and enterprise deployments based on business requirements. |

# Chapter 6 – Technology Stack

## 6.1 Introduction

The Theta Chat Bot is a modular AI-powered customer support solution that integrates with the existing Theta Technolabs website without requiring changes to the current Webflow implementation. It follows a service-oriented architecture where the frontend communicates with a FastAPI backend through REST APIs. The backend manages request validation, knowledge retrieval, prompt construction, LLM interaction, and response generation.

The technology stack was selected against four criteria:

- **Cost** – Minimize development, deployment, and operational expenses.
- **Control** – Maintain ownership of application data and infrastructure.
- **Development Velocity** – Enable rapid development, testing, and deployment.
- **Scalability** – Support future enhancements such as multilingual capabilities, analytics, and increased user traffic without requiring major architectural changes.

The system is organized into four independent layers — Frontend, Backend, AI & Knowledge Retrieval, and Infrastructure — so each layer can be maintained, scaled, or replaced on its own. Concretely, the architecture is designed to reuse the existing Webflow website, keep infrastructure lightweight and cost-effective, support future AI provider changes, enable modular feature expansion, and simplify deployment using Docker.

## 6.2 Technology Stack Overview

The Theta Chat Bot is built on a modern, modular technology stack that emphasizes scalability, maintainability, performance, and cost-effectiveness. Each technology was chosen for its suitability for AI application development, ease of integration, community support, and flexibility for future enhancements, so individual components can be upgraded or replaced with minimal impact on the rest of the system.

**Table 6.1: Technology Stack Overview**

| Layer | Technology | Role | Reason / Consideration |
|---|---|---|---|
| Chat Interface | Custom JavaScript Widget | Provides the chatbot's user interface and communicates with the backend | Lightweight, framework-agnostic, and easily integrated into the existing Webflow website |
| Programming Language | Python | Core backend and AI application development | Rich AI ecosystem, extensive libraries, and strong community support |
| Backend | FastAPI | Handles REST APIs, request validation, AI orchestration, and business logic | High-performance asynchronous framework with native Python integration, well suited for scalable AI applications |
| AI Model | Google Gemini / OpenAI | Generates natural language responses | Modular design allows switching between LLM providers with minimal code changes |
| Embedding Model | Sentence Transformers | Generates vector embeddings for semantic search | Runs locally, eliminating API costs while supporting offline deployment and full control over embedding generation |
| Vector Store / Database | Qdrant or PostgreSQL + pgvector | Stores vector embeddings and performs semantic search for Retrieval-Augmented Generation (RAG) | Qdrant is optimized for high-performance vector search; PostgreSQL with pgvector suits deployments managing both relational and vector data in a single database |
| Knowledge Base | Website Pages + Markdown Documents | Provides company information for retrieval | Easily extendable with additional document sources without modifying the overall architecture |
| Containerization | Docker & Docker Compose | Packages and deploys application services | Ensures consistent environments across development, testing, and production |
| Reverse Proxy | Nginx | Handles HTTPS, routing, and reverse proxy | Provides secure communication and supports future load balancing |
| Version Control | Git | Source code management | Enables version tracking, collaboration, and change management |
| Hosting | Linux Server (On-Premise) | Hosts the chatbot infrastructure | Provides secure, controlled, and cost-effective deployment within the organization's infrastructure |

Four components in this table — the LLM, the vector store, and the hosting model — have more than one viable option. Rather than repeat the trade-offs here, each is resolved once, with full reasoning, in Section 6.3.

## 6.3 Technology Selection Rationale

Each major component was evaluated against realistic alternatives using performance, scalability, maintainability, operational cost, ease of deployment, and compatibility with AI-based applications — rather than picked by popularity. This section is the single source of truth for *why* each choice was made; later sections (6.4–6.6) describe *how* the chosen components fit together and are not repeated here.

**Architecture Decision — Retrieval-Augmented Generation (RAG)**

**DECISION**

Adopt RAG instead of relying solely on a Large Language Model.

**ALTERNATIVES CONSIDERED**

- Direct LLM responses without retrieval
- Manually maintained FAQ database
- Fine-tuned language model

**RATIONALE**

Grounds responses in official website content, eliminates frequent retraining, and stays synchronized with website updates.

**IMPACT**

Positive: higher accuracy, better maintainability, improved trust.
Trade-off: adds a retrieval step before generation.

### 6.3.1 Backend Framework: FastAPI

**Architecture Decision — Backend Framework**

**DECISION**

Adopt FastAPI for the backend architecture.

**ALTERNATIVES CONSIDERED**

- Flask (Synchronous by default)
- Django (Too much overhead)
- Node.js (Thinner AI/ML ecosystem)
- Java / .NET (Slower iteration for AI code)

**RATIONALE**

The workload is fundamentally I/O-bound. FastAPI's async request handling supports many concurrent chats, integrates natively with Python's deep AI ecosystem, and auto-generates API docs.

**IMPACT**

Positive: high performance, single language across ML and API, self-documenting.
Trade-off: requires team familiarity with async Python programming.

### 6.3.2 Large Language Model: Google Gemini

## Architecture Decision — Primary LLM Provider

**DECISION**

Adopt Google Gemini as the initial primary LLM, but maintain a provider-agnostic layer.

**ALTERNATIVES CONSIDERED**

- OpenAI (Used as a fallback for complex reasoning)
- Self-hosted Open Source Models (High operational overhead)

**RATIONALE**

Gemini provides an excellent balance of cost, large context window, and reasoning quality for RAG. Keeping the provider as a configuration value avoids vendor lock-in. A data-driven testing approach dictates falling back to OpenAI only if Gemini fails the quality bar for a specific query.

**IMPACT**

Positive: cost-effective, easy to swap providers, data-driven.
Trade-off: requires maintaining integration with multiple LLM APIs.

### 6.3.3 Vector Storage: Qdrant, or PostgreSQL + pgvector

**Option 1 — Dedicated Vector Database (Qdrant).** When the chatbot only requires semantic search over the knowledge base and does not need conversation logging, analytics, or an administrative dashboard, Qdrant is the recommended solution. Qdrant is purpose-built for vector similarity search and provides excellent retrieval performance with minimal infrastructure overhead.

**Option 2 — PostgreSQL with pgvector.** When the application requires conversation history, user feedback, operational metrics, API logs, or an administrative dashboard, PostgreSQL with the pgvector extension is the recommended architecture. In this approach, PostgreSQL stores both relational application data and vector embeddings using the pgvector extension within a single database system, eliminating the need for a separate vector database.

This is the same fork that determines the hosting/monitoring approach in Section 6.3.5 (Option A vs. Option B) — the choice is made once and applies consistently across both.

### 6.3.4 Embeddings: Sentence Transformers (Local)

**Architecture Decision — Local Embeddings**

**DECISION**

Run Sentence Transformers locally instead of using an external embedding API.

**ALTERNATIVES CONSIDERED**

- OpenAI / Gemini Embedding APIs (Recurring per-token costs)

**RATIONALE**

Generating embeddings locally eliminates per-token API costs, works entirely offline, keeps data within the organization's infrastructure, and guarantees vectors remain in the same semantic space for both query and document representation.

**IMPACT**

Positive: zero API cost for embeddings, strong data privacy, offline support.
Trade-off: consumes local compute resources.



### 6.3.5 Deployment & Hosting: On-Premise, With a Documented Exit Ramp

**Architecture Decision — Deployment Infrastructure**

**DECISION**

Host the chatbot infrastructure on an on-premise Linux server by default.

**ALTERNATIVES CONSIDERED**

- Managed Cloud Services (Elastic scalability, ongoing recurring costs)

**RATIONALE**

An on-premise approach avoids recurring cloud compute bills for a steady, non-spiky workload. Internal server management is treated as a one-time setup cost rather than an ongoing operational expense. If compute needs grow, the hosting model can easily shift to the cloud without redesigning the architecture.

**IMPACT**

Positive: lower long-term cost, maximum data control.
Trade-off: requires internal server management and upfront setup.

### 6.3.6 Containerization: Docker + Docker Compose

Every service — the FastAPI backend, the vector store, and Nginx — ships as a container, defined once in a **docker-compose.yml**, so "works on my machine" stops being a risk. This

gives identical environments across dev, staging, and production, a one-command startup (**docker compose up**), and simple rollback by pinning image tags.

## 6.4 Component Interaction

The Theta Chat Bot follows a modular request-processing workflow in which each component handles one stage of the chatbot lifecycle. User interactions begin through the custom JavaScript chat widget embedded in the existing Webflow website. The widget communicates with the backend through secure REST APIs exposed by the FastAPI application.

**Step-by-step request flow:**

- **Request Initiation:** The user submits a query through the JavaScript widget. The request is sent via HTTPS to Nginx.
- **Validation:** Nginx routes the request to the FastAPI backend.
- **Vectorization:** The local Sentence Transformers model converts the text into a vector embedding.
- **Knowledge Retrieval:** The Vector Database performs a semantic similarity search to identify relevant knowledge.
- **Prompt Construction:** The retrieved segments and user query are combined into a context-aware prompt.
- **AI Inference:** The LLM generates a natural language response.
- **Response Delivery:** The generated answer is sent back to the frontend widget.



Separating user interaction, business logic, knowledge retrieval, and AI inference into independent components improves maintainability, simplifies testing, enables future technology replacement, and supports incremental enhancements without major architectural changes.

flowchart TB U([User]) subgraph Frontend WF["Webflow Website / JS Widget"] end subgraph Backend N["Nginx"] API["FastAPI Backend"] PB["Prompt Builder"] end subgraph Knowledge EMB["Sentence Transformers"] DB[(Qdrant / PostgreSQL + pgvector)] end subgraph AI LLM["Google Gemini / OpenAI"] end U -->|Chat Message| WF WF -->|HTTPS| N N --> API API --> EMB EMB --> DB DB -->|Relevant Documents| PB API -->|User Query| PB PB -->|Final Prompt| LLM LLM --> API API -->|Response| WF WF --> U %% Styling classDef process fill:#1c5d8c,stroke:#0d2b45,stroke-width:2px,color:#ffffff; classDef highlight fill:#fdf6ea,stroke:#e8a33d,stroke-width:2px,color:#b5791c; classDef startend fill:#0d2b45,stroke:#e8a33d,stroke-width:3px,color:#ffffff; class U startend; class WF,N,API,PB process; class EMB,DB,LLM highlight; linkStyle default stroke:#0d2b45,stroke-width:2px;

**Figure 6.1: Component Interaction Architecture**

## 6.5 APIs Used

| API | Type | Purpose |
|---|---|---|
| **Internal — /api/chat** | REST (FastAPI) | The only endpoint the frontend widget calls; accepts a user message, returns the generated answer + sources |
| **Gemini API** | External | Primary LLM — generates the natural-language response from the assembled prompt |
| **OpenAI API** *(optional)* | External | Fallback LLM, used case-by-case when a Gemini response fails the quality bar during testing |
| **Gemini Embedding API** | External | Alternative embedding generator to Sentence Transformers, if a hosted embedding service is preferred over running locally |
| **Qdrant REST API** | Internal (self-hosted) | Stores vectors, performs similarity search; called by the backend, never by the frontend |

> This is a basic example of the API surface at the current stage — more internal endpoints (e.g. for feedback, session handling, or an admin dashboard) will be added as the backend grows.



## 6.6 Frontend: Integrating with Webflow Using an Embedded Widget

The Theta Technolabs website is already built and maintained on Webflow. To preserve it while adding chatbot functionality, the chatbot is implemented as a lightweight embedded JavaScript widget rather than a rebuild — a single **<script>** embed, no changes to the existing Webflow structure or design.

This gets the project:

- Integration with the existing Webflow website without structural changes.
- Simple deployment via one **<script>** tag.
- Preservation of the existing design and content-management workflow.
- Independent customization of the chatbot's appearance — colors, position, branding — separate from the site itself.
- The ability to update the chatbot later without touching the website implementation.

# Chapter 7 – API Design & Communication Flow

## 7.1 Overview

The Theta Chat Bot follows a RESTful API architecture, where the chatbot widget communicates with the FastAPI backend through secure HTTPS endpoints.

The backend acts as the central orchestration layer responsible for validating requests, retrieving relevant knowledge, communicating with the Large Language Model (LLM), and returning structured responses to the frontend.

All communication between components is performed using JSON payloads, making the APIs lightweight, language-independent, and easy to integrate with existing or future frontend applications.

## 7.2 API Communication Architecture

The communication between system components follows the sequence below:

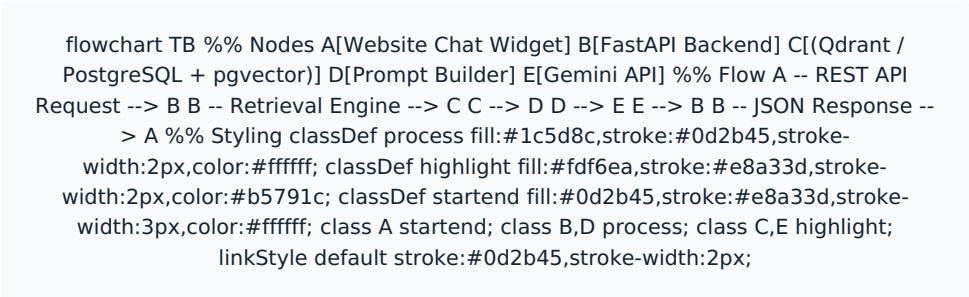


**Figure 7.1: API Communication Architecture**

Each component has a clearly defined responsibility, ensuring loose coupling and easier maintenance.

## 7.3 API Endpoints

**Note:** The endpoints listed below are representative examples of proposed APIs. Actual endpoints, path structures, and payload specifications may be expanded or adjusted based on implementation needs.

Future administrative and internal endpoints may include analytics, session control, health diagnostics, knowledge synchronization, and dashboard APIs.

## 7.4 Request Flow

When a visitor submits a message, the frontend sends a POST request to the chatbot API endpoint. The request body uses a structured JSON schema to pass the user message and session context to the backend.

> **Note:** The request schema shown below is indicative. Field names, optional parameters, and request structure may be adjusted during implementation to fit final integration requirements.

**POST /api/v1/chat — Request Body Schema**

## 7.5 Backend Processing Flow

Upon receiving the request, the backend performs the following operations:

1. Validate the request payload.

2. Verify session information.
3. Sanitize user input.
4. Generate the query embedding.
5. Search the vector database.
6. Retrieve the Top-K relevant content chunks.
7. Build the final prompt.
8. Send the prompt to the configured LLM.
9. Validate the generated response.
10. Return the final response to the frontend.

Each step is executed sequentially to ensure consistent and reliable responses.

## 7.6 Response Structure

The backend returns a standardized JSON response envelope. Every successful response includes a status indicator, the AI-generated answer, source references used to construct the response, a confidence classification, and backend processing metadata.

> **Note:** The response schema shown below is a representative sample. The exact field names, confidence scoring levels, and source reference structure may be refined during implementation and adjusted at runtime without breaking the core API contract.

**Success Response — Field Schema**

Source references are included to increase transparency and allow users to verify the information directly on the Theta Technolabs website.

## 7.7 Error Handling

The API returns a standardized error envelope whenever a request cannot be completed successfully. All error responses share the same top-level structure as success responses to simplify client-side parsing.

> **Note:** Error codes and messages are indicative. Additional error codes may be introduced at runtime as new edge cases are identified during testing and production operation.

**Error Response — Field Schema**

**Common Error Codes**

Providing standardized error codes simplifies debugging and future monitoring.

## 7.8 HTTP Status Codes

## 7.9 Session Management

Although the chatbot is publicly accessible and does not require user authentication, temporary browser sessions are maintained to support conversational context.

Each visitor receives a unique session identifier when opening the chatbot.

The session identifier enables:

- Context-aware follow-up questions
- Temporary conversation memory
- Improved response quality
- Analytics (optional)

No personally identifiable information (PII) is required to create or maintain a session.

## 7.10 Security Considerations

To protect the chatbot APIs from misuse, the backend implements multiple validation and security mechanisms.

These include:

- HTTPS communication
- Input validation
- Request sanitization
- Maximum message length validation
- API rate limiting (future enhancement)
- CORS configuration
- Environment-based API key management
- Secure server-side communication with LLM providers

Sensitive API credentials are never exposed to the frontend.

## 7.11 API Versioning

All public endpoints are versioned using URI-based versioning.

Example: **/api/v1/chat**

Versioning allows future API enhancements without breaking compatibility with existing frontend integrations.

## 7.12 Key Design Decisions

# Chapter 8 – Frontend Integration

## 8.1 Overview

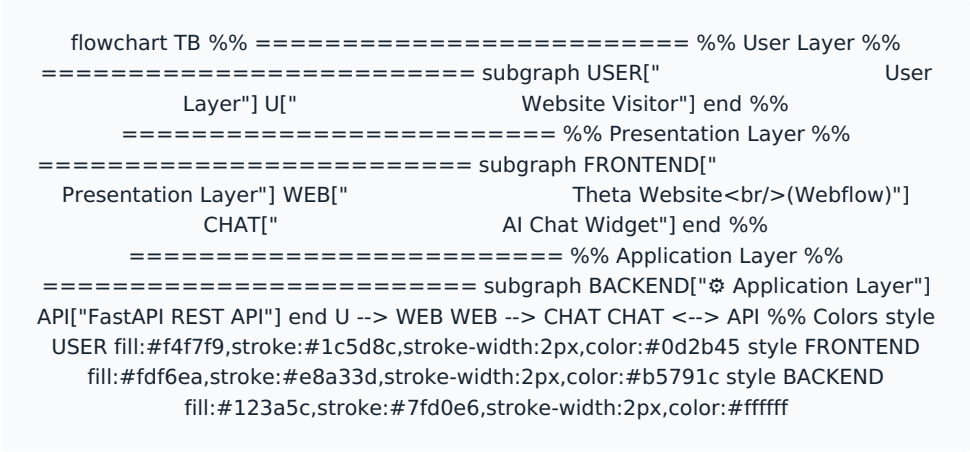


**Figure 8.1: Frontend Integration Overview**

The Theta Chat Bot will be integrated into the existing Theta Technolabs website without requiring any major modifications to the current frontend architecture. Since the website is developed using a no-code platform (Webflow), the chatbot will be embedded using a lightweight JavaScript widget that communicates with the FastAPI backend through secure REST APIs.

This approach minimizes development effort while ensuring that the chatbot provides a seamless and consistent user experience across all website pages.

The frontend is responsible for the following functions:

- Displaying the chatbot interface
- Capturing user queries
- Managing the current browser session
- Sending requests to the backend
- Displaying AI-generated responses
- Providing source references (when available)
- Handling loading, error, and fallback states

> The frontend contains **no AI logic**. All retrieval, prompt construction, and response generation are handled exclusively by the backend.

## 8.2 Frontend Architecture & Widget Placement

The chatbot frontend consists of three primary components:

| Component | Description | Responsibilities |
|---|---|---|
| **Chat Launcher** | A floating action button displayed on every page of the website. | Open/close chatbot window, preserve session state |
| **Chat Window** | The main conversation interface. | Display conversation history, accept user input, display typing indicators, render AI responses, show source references, display quick action buttons and fallback options |

| | | |
|---|---|---|
| **API Communication Layer** | Responsible for communication with the backend. | Send REST API requests, receive JSON responses, handle network failures, retry failed requests (future enhancement) |

**Widget Placement Options:** The chatbot interface can be triggered either via the main navigation menu (e.g., "Chat with AI" button) or through a persistent floating widget in the bottom-right corner of the screen, as illustrated below:

## 8.3 Frontend Request Flow

Every user interaction follows the same communication flow:

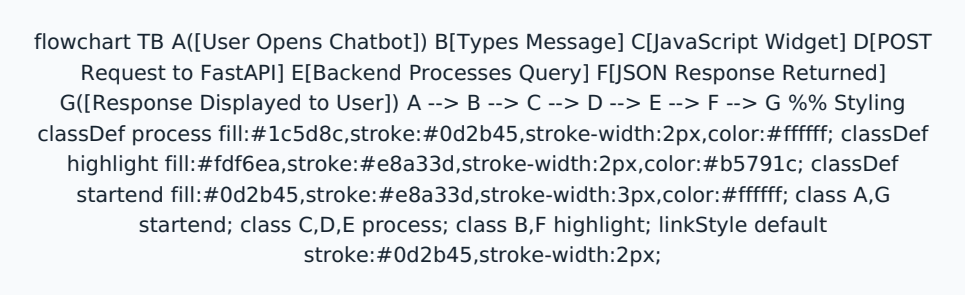


**Figure 8.3: Frontend Request Flow**

This lightweight communication model keeps the frontend simple while allowing the backend to manage all business logic.

## 8.4 User Journey

The following sequence describes a typical user interaction:

1. Visitor opens the Theta Technolabs website.
2. The chatbot launcher appears at the bottom-right corner of the screen.
3. The visitor clicks the chatbot icon.
4. The chatbot displays a welcome message: *"Hello! I'm the Theta AI Assistant. How can I help you today?"*
5. The visitor enters a question.
6. The frontend sends the request to the backend.
7. The chatbot displays a typing indicator while the backend processes the request.
8. The AI-generated response is displayed.
9. If available, the chatbot displays a reference to the related website page.
10. The user can continue the conversation or close the chatbot.

## 8.5 User Interface States

To provide a smooth user experience, the chatbot supports multiple interface states:

| State | Description |
|---|---|
| Welcome | Initial greeting when chatbot opens |
| Idle | Waiting for user input |
| Typing | User entering a message |
| Loading | Backend is processing the request |
| Response | AI-generated answer displayed |
| Error | Temporary server or network issue |
| No Result | Information not found in the knowledge base |

| | |
|---|---|
| Out of Scope | Question is outside the chatbot's supported domain |

Each state provides clear visual feedback to improve usability.

## 8.6 Response Presentation

Every chatbot response should follow a consistent format. The response may include:

- AI-generated answer
- Related service or page reference
- Relevant website link (if available)
- Contact recommendation (when required)

> **Example Response**
>
> **AI Response:** Theta Technolabs provides Artificial Intelligence Development, Machine Learning, Computer Vision, and Conversational AI solutions.
>
> **Related Page:** AI Development Services — View Details →

This structure improves transparency and encourages users to explore the website further.

## 8.7 Fallback User Experience

If the chatbot cannot confidently answer a question, it should provide a helpful fallback experience instead of generating uncertain information.

> **Scenario 1 – Low Confidence**
>
> **Message:** "I couldn't find enough verified information to answer your question accurately. You may find more details on our website or contact our team for further assistance."
> **Available Actions:** View Related Services · Contact Us · Submit Inquiry

> **Scenario 2 – Out-of-Scope Query**
>
> **Message:** "I'm designed to answer questions related to Theta Technolabs, including our services, technologies, portfolio, careers, and company information. I can't assist with unrelated topics."
> **Available Actions:** Visit Services · Contact Sales · Submit Inquiry

This approach maintains transparency while guiding users toward the appropriate next step.

## 8.8 Responsive Design

The chatbot interface should provide a consistent experience across all supported devices:

- Desktop browsers
- Tablets
- Mobile devices

Key design considerations:

- Responsive layout
- Scrollable conversation history
- Touch-friendly controls
- Optimized input field
- Adaptive chatbot window size

## 8.9 Accessibility Considerations

The chatbot should follow general accessibility best practices to ensure usability for a wider audience. Recommended considerations include:

- Keyboard navigation support
- Screen reader compatibility
- Sufficient color contrast
- Readable typography

- Visible focus indicators
- Accessible button labels

Implementing these practices improves the overall user experience and aligns with modern web accessibility standards.

## 8.10 Future User Experience Enhancements

The modular frontend architecture allows future improvements without changing the backend. Potential enhancements include:

- Voice-based interaction
- File upload support
- Multi-language interface
- Rich cards and service recommendations
- Appointment booking
- Conversation export
- Suggested Questions (On initial load)

## 8.11 Key Design Decisions

| Decision | Justification |
|---|---|
| Embedded JavaScript widget | Enables seamless integration with the existing website without redevelopment. |
| Backend-driven AI processing | Keeps the frontend lightweight and secure by avoiding AI logic in the browser. |
| Session-based conversation | Improves conversational continuity while remaining anonymous. |
| Confidence-based fallback | Prevents misleading responses and improves user trust. |
| Responsive design | Ensures consistent experience across desktop and mobile devices. |
| Accessibility support | Improves usability for all users and aligns with modern web standards. |

# Chapter 9 – AI Guardrails Strategy

## 9.1 Overview

To ensure the Theta Chat Bot provides accurate, reliable, and business-appropriate responses, a comprehensive set of AI guardrails is implemented. These guardrails define how the chatbot processes user queries, retrieves information, generates responses, and handles situations where sufficient information is unavailable.

The primary objective is to ensure that every response is based on the approved Theta Technolabs knowledge base while preventing hallucinations, prompt injection attacks, and responses outside the chatbot's intended scope.

## 9.2 Guardrail Workflow

The chatbot applies multiple validation steps before returning a response to the user.



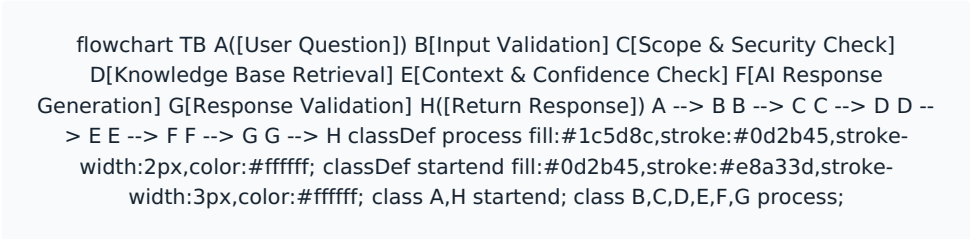


**Figure 9.1: Guardrail Workflow**

## 9.3 Scope Guardrail

The chatbot is designed exclusively to answer questions related to Theta Technolabs. Before processing a query, the system determines whether it falls within the supported business scope.

Supported topics include: Company information, Services, Technologies, AI solutions, Mobile/web development, Careers, Contact info, Blogs and case studies. Questions unrelated to Theta Technolabs are treated as out-of-scope.

> **EXAMPLE**
>
> **User:** Who won the FIFA World Cup?
> **Bot:** I'm designed to answer questions related to Theta Technolabs and its services. For information outside this scope, please refer to an appropriate external source.

## 9.4 Knowledge Base Guardrail

Every response generated by the chatbot must be based on information retrieved from the approved Theta Technolabs knowledge base. If no relevant content is available, the chatbot does not attempt to guess the answer.

## 9.5 Confidence Guardrail

After retrieving relevant content from Qdrant, the system evaluates the similarity score to determine whether there is enough confidence to answer the question.

| Similarity Score | Action |
|---|---|
| ≥ 0.80 | Generate a confident response using the retrieved context. |
| 0.60 – 0.79 | Generate a response using the available context while limiting unsupported details. |
| < 0.60 | Treat the query as unsupported and return a fallback response. |

This validation reduces the risk of generating incorrect or misleading information. *(See Section 4.9 — Response Validation for the corresponding system behavior for each confidence level).*

## 9.6 Prompt Guardrail

The AI model is guided by a predefined system prompt that defines its behavior and limitations, instructing it to answer only using retrieved context and decline out-of-scope queries.



> **EXAMPLE SYSTEM PROMPT**
>
> You are the official Theta Technolabs assistant. Answer only using the provided context. Do not invent information. Do not answer unrelated questions. If the requested information is unavailable, politely state that you cannot find it and recommend visiting the official website.

## 9.7 Hallucination Prevention Guardrail

The chatbot must never fabricate or assume information that is not available in the approved knowledge base (e.g., pricing, commitments, unlisted services). If unavailable, a standard fallback message is returned.

## 9.8 Website Link Guardrail & Response Formatting

Whenever possible, responses include relevant webpage source links for transparency. Structure follows: (1) Direct answer, (2) Concise explanation, (3) Link to relevant webpage (150–250 words).

## 9.9 Security Guardrail

The chatbot safely handles requests attempting to manipulate behavior or expose system information (e.g., prompt injection, revealing API keys or system instructions).

## 9.10 Unsupported Query Handling

Topics unrelated to Theta Technolabs (sports, weather, politics, general coding) are politely declined with a standard scope clarification.

## 9.11 Fallback Strategy

The chatbot follows a structured decision process whenever sufficient information is not available.

flowchart TB A([User Question]) B[Search Knowledge Base] C{Relevant Context Available?} D[Generate Response] E{Is Query Related to Theta?} F([Suggest Contact or Official Page]) G([Politely Decline]) A --> B B --> C C --|Yes|--> D C --|No|--> E E --|Yes|--> F E --|No|--> G classDef process fill:#1c5d8c,stroke:#0d2b45,stroke-width:2px,color:#ffffff; classDef startend fill:#0d2b45,stroke:#e8a33d,stroke-width:3px,color:#ffffff; classDef decision fill:#fdf6ea,stroke:#e8a33d,stroke-width:2px,color:#b5791c; class A,D,F,G startend; class B process; class C,E decision;



**Figure 9.2: Fallback Strategy Decision Process**

## 9.12 Benefits of the Guardrail Strategy

- Ensures responses are grounded strictly in approved company information.
- Prevents hallucinations and protects against prompt injection attacks.
- Maintains a consistent, professional communication style with verifiable web links.
- Delivers predictable fallback handling for out-of-scope or low-confidence queries.

# Chapter 10 – Operational Dashboard & Analytics

## 10.1 Overview

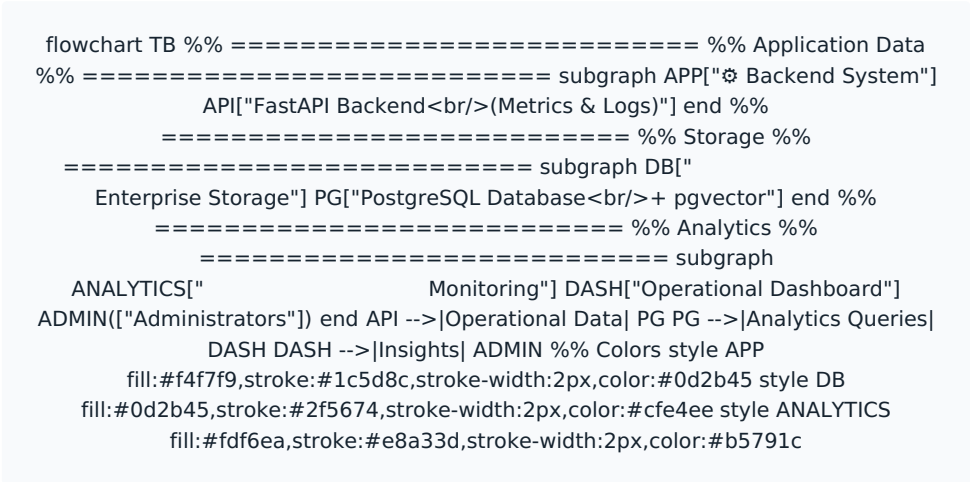


**Figure 10.1: Dashboard Architecture Overview**

While the chatbot can operate as a standalone lightweight application, organizations often require visibility into system usage, AI performance, operational costs, and user interactions. To address these needs, an optional Operational Dashboard is proposed.

The dashboard provides centralized monitoring and analytics, enabling administrators to understand how the chatbot is performing, identify knowledge gaps, monitor API usage, and make informed decisions about future improvements.

> The dashboard is **optional** and is recommended for production deployments where long-term monitoring and business insights are required.

## 10.2 Dashboard Architecture

When the dashboard is enabled, PostgreSQL with the pgvector extension is recommended as the primary database. The database stores both vector embeddings and operational data, allowing semantic search and analytics to coexist within a single platform.

Operational data may include:

- Conversation metadata
- User feedback
- API requests
- Token usage
- Response time
- Error logs
- Knowledge synchronization history
- Dashboard configuration

This architecture reduces infrastructure complexity by avoiding multiple databases for related operational data.

## 10.3 Dashboard Modules

The Operational Dashboard is organized into five functional modules:

#### A. Conversation Analytics

Provides insights into chatbot usage: **Total conversations, daily/weekly/monthly volumes, average conversation length, average response time, and active sessions.** These metrics help evaluate chatbot adoption and user engagement.

#### B. Knowledge Analytics

Measures how effectively the knowledge base serves user queries: **Most accessed pages, most searched services, popular technologies, frequently asked questions, low-confidence responses, unanswered queries, and out-of-scope requests.** These insights identify areas where website content may need improvement.

#### C. AI Usage Analytics

Monitors LLM usage and operational cost: **Total API requests, input tokens, output tokens, total token consumption, estimated API cost (daily/monthly), average processing time, and active LLM provider.** This helps optimize AI usage and control operational expenses.

#### D. User Feedback Analytics

Analyzes user satisfaction (when feedback is enabled): **Helpful/Not Helpful ratios, feedback trends, user comments, and most frequently disliked responses.** Supports continuous improvement of chatbot quality.

#### E. System Health Monitoring

Provides operational visibility: **API availability, error rate, failed requests, average latency, synchronization status, Docker container health, database connectivity, and storage utilization.** Helps administrators detect issues before they impact users.

## 10.4 Dashboard Database Design

When PostgreSQL is used, the following logical tables are recommended:

| Table | Purpose |
|---|---|
| conversations | Session-level conversation information |
| messages | Individual chatbot and user messages |
| feedback | User feedback records |
| api_usage | API request statistics and token usage |
| sync_history | Knowledge synchronization history |
| dashboard_settings | Dashboard configuration |
| error_logs | Application and API errors |

Additional tables can be introduced as future requirements evolve.

## 10.5 Example Dashboard KPIs

The dashboard homepage can present a high-level overview of system performance:

| KPI | Description |
|---|---|
| Total Conversations | Total number of chatbot conversations |
| Messages Today | Number of messages processed today |
| Average Response Time | Mean response latency |
| Knowledge Coverage | Percentage of queries answered using indexed content |
| API Cost | Estimated LLM usage cost |
| Positive Feedback | Percentage of helpful responses |
| Failed Requests | Number of failed API requests |

| | |
|---|---|
| Last Synchronization | Timestamp of the latest knowledge update |

## 10.6 Dashboard Benefits

**Business Benefits:**

- Measure chatbot adoption
- Understand user interests
- Identify popular services
- Improve customer engagement
- Support business decision-making

**Technical Benefits:**

- Monitor AI performance
- Detect knowledge gaps
- Optimize operational costs
- Track synchronization status
- Improve system reliability

## 10.7 Future Dashboard Enhancements

The modular dashboard architecture allows future expansion without redesign. Potential enhancements include:

- Real-time analytics
- Interactive charts
- Geographic usage reports
- Device and browser statistics
- CRM integration
- Lead generation analytics
- Export to Excel or PDF
- Email reports
- Role-based access control
- AI-powered operational summaries

## 10.8 Dashboard Availability

| Deployment | Dashboard Included |
|---|---|
| Lightweight (Qdrant) | Not Included |
| Enterprise (PostgreSQL + pgvector) | Included |

Organizations that require only chatbot functionality can deploy the lightweight architecture without operational analytics. As business requirements evolve, the enterprise architecture can be adopted without changing the overall chatbot design.

## 10.9 Key Design Decisions

| Decision | Justification |
|---|---|
| PostgreSQL + pgvector for enterprise deployments | Supports both semantic search and operational analytics within a single database platform. |
| Modular dashboard architecture | Allows independent enhancement of analytics features. |
| Separate analytics modules | Simplifies monitoring by organizing metrics into functional categories. |
| KPI-based monitoring | Provides quick visibility into chatbot health and business performance. |
| Optional deployment | Keeps the MVP lightweight while enabling enterprise-grade observability when required. |

# Chapter 11 – Security & Bot Prevention

## 11.1 Overview

Although the Theta Chat Bot is publicly accessible and does not require user authentication, security remains a critical aspect of the overall system design. The chatbot communicates with external AI services, processes user input, and retrieves information from the company's knowledge base. Therefore, appropriate security controls must be implemented to ensure system reliability, protect sensitive configurations, and prevent misuse.

The proposed security strategy follows a **defense-in-depth approach**, where multiple layers of protection work together to reduce potential risks.

## 11.2 Security Objectives

The primary security objectives of the chatbot are:

- Protect backend services from unauthorized access
- Secure communication between all system components
- Prevent abuse of public APIs
- Protect AI provider API credentials
- Maintain the integrity of the knowledge base
- Ensure reliable and uninterrupted chatbot operation

> Since the chatbot answers questions **only about Theta Technolabs**, it is intentionally restricted to information available in the approved knowledge base.

## 11.3 Communication Security

All communication between the website, backend, vector database, and external AI provider should use encrypted channels.

Recommended practices include:

- HTTPS for all frontend-to-backend communication
- Secure communication between internal services where applicable
- Automatic redirection from HTTP to HTTPS

## 11.4 API Security

Because the chatbot APIs are publicly accessible, they should include multiple validation mechanisms. Recommended controls:

- Request validation
- Input sanitization
- Maximum message length limits
- Request timeout configuration
- Rate limiting (recommended for production)
- Cross-Origin Resource Sharing (CORS) configuration
- API versioning

## 11.5 API Key Management

The chatbot communicates with external AI providers such as Google Gemini and, potentially, OpenAI. API credentials must never be exposed to the frontend. Recommended practices:

1. Store API keys in environment variables.

2. Never hardcode credentials in source code.
3. Rotate API keys periodically.
4. Restrict API key permissions where supported.
5. Maintain separate credentials for development, testing, and production environments.

All AI requests should originate from the backend to ensure credentials remain protected.

## 11.6 Knowledge Base Protection

The chatbot should retrieve information only from the approved Theta Technolabs knowledge base. To maintain data integrity:

- Only indexed website content should be used
- Unauthorized data sources should not be queried
- Content updates should follow the synchronization pipeline
- Metadata should be preserved for traceability
- Version information should be maintained for indexed content

## 11.7 Privacy Considerations

The chatbot is designed to answer general questions about Theta Technolabs and does not require users to register or provide personal information. By default:

- No login required
- No PII collected intentionally
- Temporary session identifiers only — expire after inactivity
- Future CRM/forms integration will require a privacy review



## 11.8 Logging & Audit Trail

Application logging is important for troubleshooting, monitoring, and operational analysis. The system should record:

- API requests
- Processing time
- Error events
- Synchronization events
- AI provider response status
- System warnings

> Sensitive information such as API keys, authentication tokens, or confidential configuration values must **never** be written to application logs.

## 11.9 Security Risks & Mitigation

| Potential Risk | Mitigation Strategy |
|---|---|
| Prompt injection attempts | Restrict responses to retrieved knowledge base content and enforce system prompt rules. |
| API abuse | Implement rate limiting and request validation. |
| Unauthorized API access | Expose only required public endpoints and validate all requests. |
| API key exposure | Store credentials securely on the server using environment variables. |
| Invalid or malicious input | Sanitize and validate all user input before processing. |
| Hallucinated responses | Use Retrieval-Augmented Generation (RAG) with confidence-based response validation. |
| Service outage | Return a user-friendly fallback message and log the incident for investigation. |

## 11.10 Bot Prevention & CAPTCHA Strategy (Future Scope)

*Note: The following CAPTCHA and rate-limiting features are planned for future deployment and are not included in the initial release.*

To prevent malicious bots and automated scripts from exhausting the AI provider's API budget (Financial DoS) or skewing operational analytics, an invisible CAPTCHA mechanism (e.g., Google reCAPTCHA v3 or Cloudflare Turnstile) is implemented on the frontend.

**Technical Flow:**

1. **Frontend Generation:** The JavaScript widget silently evaluates browser behavior and generates a temporary `captcha_token` without requiring the user to solve visual puzzles.
2. **API Request:** This token is submitted along with the user's message in the `POST /api/v1/chat` request.
3. **Backend Verification:** The FastAPI backend validates the token with the CAPTCHA provider before initializing the retrieval pipeline.
4. **Decision:** If the score indicates a bot, the request is immediately rejected (`403 Forbidden`). **The LLM is never called, and no API costs are incurred.**

> **Second Layer of Defense:** Even if a user bypasses the CAPTCHA, strict IP-based rate limiting (e.g., max 20 messages per 10 minutes) acts as a fail-safe against manual or distributed spam attacks.

## 11.11 Future Security Enhancements

As the chatbot evolves, additional security features may be introduced:

- Web Application Firewall (WAF)
- Advanced rate limiting policies
- Audit log management
- Secrets management using a vault solution
- Security monitoring and alerting

## 11.12 Key Design Decisions

| Decision | Justification |
|---|---|
| HTTPS for all communications | Protects data in transit and ensures secure communication. |
| Backend-only AI communication | Prevents exposure of AI provider credentials to the client. |
| Environment-based configuration | Improves security and simplifies deployment across environments. |
| Invisible CAPTCHA integration | Protects against API budget drain without harming the user experience. |
| RAG-based response generation | Grounds responses in approved website content. |
| Temporary session identifiers | Supports conversational continuity without requiring user authentication. |
| Optional enterprise security enhancements | Allows the security model to evolve alongside future business requirements. |

# Chapter 12 – Deployment Strategy

## 12.1 Overview

The chatbot is containerized using Docker for consistent, portable deployment across development, testing, and production. Initial deployment is on-premise, hosted within company infrastructure — giving full control, predictable costs, and easier compliance. The architecture remains cloud-ready for future migration to AWS, Azure, or GCP with minimal changes.

## 12.2 Environments

| Environment | Purpose |
|---|---|
| Development | Feature development and local testing |
| Testing / UAT | Validation before production release |
| Production | Live deployment for website visitors |

## 12.3 Containerized Components

| Container | Responsibility |
|---|---|
| Nginx | Reverse proxy, SSL termination, routing |
| FastAPI | REST APIs and chatbot business logic |
| Qdrant (Lightweight) | Vector storage and semantic search |
| PostgreSQL + pgvector (Enterprise) | Vector storage, analytics, operational data |
| pgAdmin (Optional) | PostgreSQL administration |
| Monitoring Stack (Future) | Metrics collection and visualization |

## 12.4 Deployment Options

**Option A – Lightweight**

**Stack:** FastAPI + Qdrant + Nginx + Docker
**Best for:** Minimal resources, fast setup, low maintenance. No conversation storage required.

**Option B – Enterprise**

Adds **PostgreSQL + pgvector + pgAdmin** for conversation analytics, token/cost tracking, feedback analysis, and an operational dashboard.



## 12.5 Configuration & Backup

All configuration (API keys, DB connection strings, LLM provider, ports, logging, session timeout) is externalized via environment variables — never hardcoded. Backups run on a schedule for PostgreSQL/Qdrant data, config files, and source code, with periodic restore testing.

> Since the knowledge base is derived from the live website, embeddings can be **fully regenerated** if lost — the knowledge base is not a single point of failure.

## 12.6 Deployment Workflow

1. Pull the latest source code.
2. Update configuration files and environment variables.
3. Build Docker images.
4. Start containers with `docker compose up`.
5. Verify database and AI provider connectivity.
6. Run API health checks (`/api/v1/health`).
7. Trigger knowledge base synchronization.
8. Validate chatbot functionality end-to-end.
9. Release to production traffic.

## 12.7 Key Design Decisions

| Decision | Justification |
| --- | --- |
| Docker-based deployment | Consistency and portability across environments |
| On-premise hosting | Infrastructure control and long-term cost efficiency |
| Environment-based config | Keeps secrets out of the codebase |
| Dual deployment architecture | Supports lightweight and enterprise needs without redesign |

# Chapter 13 – Monitoring & Logging

## 13.1 Overview

Monitoring and logging are essential for maintaining the reliability, performance, and operational health of the Theta Chat Bot. Continuous visibility into application behavior helps administrators catch issues early, optimize performance, and improve response quality over time.

The monitoring strategy covers three primary areas: Application Health, AI Performance, and Knowledge Base Freshness.

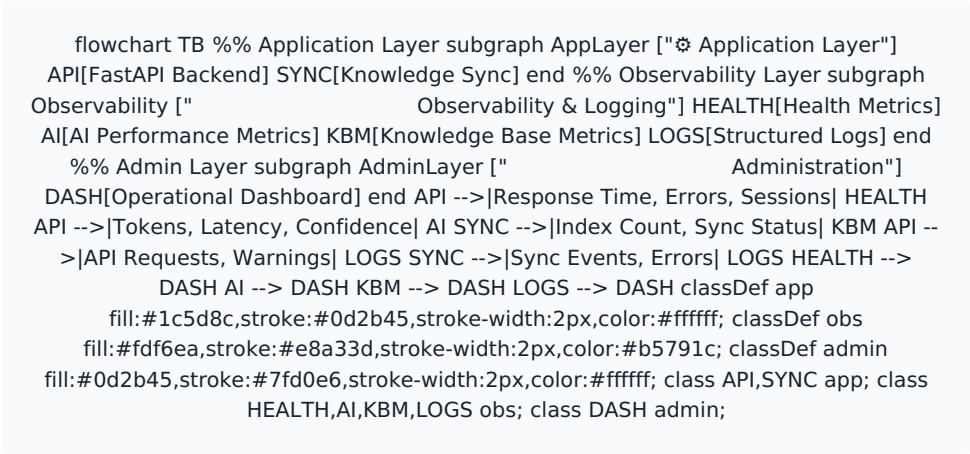


**Figure 12.1: Monitoring and Observability Strategy**

## 13.2 Monitoring Objectives

The monitoring system is designed to:

- Ensure high application availability.
- Detect operational issues early.
- Monitor chatbot performance.
- Measure AI response quality.
- Track operational costs.
- Support troubleshooting and maintenance.

## 13.3 Application Health



Monitoring application health involves tracking several key metrics.

| Metric | Purpose |
|---|---|
| API Availability | Verify backend uptime |
| Response Time | Measure request latency |
| Error Rate | Detect application failures |
| Active Sessions | Monitor current usage |
| Request Volume | Measure traffic patterns |

## 13.4 AI Performance

AI performance is tracked to evaluate cost and quality. Metrics include: total AI requests, average generation time, token consumption, estimated API cost, retrieval confidence, and LLM provider utilization.

## 13.5 Knowledge Base Monitoring

| Metric | Description |
|---|---|
| Total Indexed Pages | Pages currently indexed |
| Total Content Chunks | Semantic chunks in the KB |
| Last Synchronization | Most recent successful update |
| Failed Synchronizations | Sync errors |
| Knowledge Version | Current indexed version |

## 13.6 Logging Strategy

Logs capture API requests, AI requests, synchronization events, warnings, and errors — using standard levels (INFO, WARNING, ERROR, CRITICAL) to keep noise low while enabling fast troubleshooting. API keys, tokens, and other credentials must never be written to logs.

## 13.7 Key Design Decisions

| Decision | Justification |
|---|---|
| Continuous health monitoring | Ensures reliable availability |
| Structured logging | Simplifies troubleshooting |
| AI performance monitoring | Supports cost and quality optimization |
| Knowledge sync monitoring | Ensures responses stay current |

# Chapter 14 – AI System Validation & Testing

## 14.1 Overview

Unlike traditional software, the Theta Chat Bot requires validating both software functionality *and* AI response quality. Because it uses a RAG architecture, testing extends beyond API checks to retrieval accuracy, prompt effectiveness, chunking quality, and response reliability — combining manual evaluation with technical verification across the full pipeline.

## 14.2 Testing Scope

| Component | Validation Focus |
|---|---|
| Website Content Processing | Successful extraction and cleaning |
| Chunking Pipeline | Semantic chunk boundaries are appropriate |
| Embedding Generation | Embeddings generated successfully |
| Vector Database | Semantic retrieval accuracy |
| Prompt Builder | Correct context injection and formatting |
| Large Language Model | Response quality and consistency |
| FastAPI Backend | API functionality and business logic |
| Frontend Integration | End-to-end user experience |



## 14.3 Manual Retrieval Validation

1. **Prepare test questions** Covering all major website sections.
2. **Submit each question** Through the chatbot.
3. **Inspect retrieved chunks** Before LLM generation.
4. **Verify relevance** Confirm chunks contain what's needed to answer correctly.
5. **Record issues** Log retrieval problems for follow-up.

## 14.4 Chunking & Prompt Validation

**Chunking:** reviewed manually for logical paragraph boundaries, complete ideas, no abrupt breaks, minimal duplication, and appropriate size. Poor chunking degrades response quality even when retrieval works correctly.

**Prompt:** validated for scope adherence, correct use of retrieved context, proper handling of insufficient information, and consistent fallback behavior. Any prompt change triggers full regression testing before deployment.

## 14.5 Response Accuracy Criteria

| Criterion | Description |
|---|---|
| Accuracy | Response matches website content |
| Completeness | No important information omitted |
| Relevance | Directly addresses the question |

| | |
|---|---|
| Consistency | Similar questions get consistent answers |
| Grounding | Supported by retrieved content |
| Readability | Clear and easy to understand |

## 14.6 Regression & Performance Testing

Regression testing runs after any prompt modification, content update, knowledge sync, embedding/LLM provider change, or backend change — using a standardized reference question set covering every major website category (services, technologies, portfolio, careers, contact, general inquiries) for objective before/after comparison.

Performance is measured via average response time, retrieval latency, LLM generation time, end-to-end processing time, and concurrent request handling.

## 14.7 Error & Fallback Testing

Tested against unrelated questions, incomplete input, invalid input, empty messages, and no-match queries. Expected behavior: never fabricate information — inform the user it's unavailable and recommend the relevant page or Contact/Inquiry channel.

## 14.8 Acceptance Criteria

The chatbot is deployment-ready when: content is fully indexed, retrieval consistently returns relevant results, chunk and prompt quality are verified, responses accurately reflect website information, fallback behavior works correctly, performance meets targets, and no critical defects remain.



## 14.9 Evaluation Scorecard

Rather than a simple pass/fail, each test question is scored across pipeline stages — so a failure points to exactly where it occurred, not just that "the answer was wrong."

| Test ID | Question | Retrieved? | Chunk Quality | Prompt OK? | Accuracy | Grounded? | Time | Status |
|---|---|---|---|---|---|---|---|---|
| TC-001 | AI services offered? | ✓ | 5 | ✓ | 5 | ✓ | 1.2s | Pass |
| TC-002 | Flutter app development? | ✓ | 4 | ✓ | 5 | ✓ | 1.0s | Pass |
| TC-003 | Who is the CEO of Google? | N/A | N/A | ✓ | N/A | ✓ (Fallback) | 0.8s | Pass |

**KEY DESIGN DECISIONS**

Manual retrieval and chunk review provide direct verification before AI response quality is judged. Full regression testing after any prompt change catches unintended behavior shifts. A standardized dataset enables objective version-to-version comparison. Validation continues after deployment — not just before it — as the knowledge base evolves.

# Chapter 15 – Risk Assessment & Mitigation

## 15.1 Overview

Every software solution introduces technical and operational risks that should be identified during the design phase. Evaluating these risks early enables the team to define appropriate mitigation strategies and reduces the likelihood of unexpected issues during deployment or production.

The following table summarizes the primary risks identified for the Theta Chat Bot project.

## 15.2 Risk Register

| Risk | Impact | Probability | Mitigation Strategy |
|---|---|---|---|
| Website content becomes outdated | High | Medium | Implement event-driven incremental synchronization whenever content is published or updated. |
| AI generates unsupported information (hallucination) | High | Medium | Use Retrieval-Augmented Generation (RAG) and restrict responses to retrieved website content. |
| API provider downtime | High | Low | Display a user-friendly fallback message and monitor provider availability. Future support for multiple LLM providers can improve resilience. |
| Increased API costs | Medium | Medium | Monitor token usage, analyze operational costs, and optimize prompts and retrieval strategies. |
| Knowledge retrieval returns low-confidence results | Medium | Medium | Apply confidence thresholds and direct users to the Contact or Inquiry page when sufficient information is unavailable. |
| Vector database corruption or failure | High | Low | Maintain regular database backups and validate restoration procedures. |
| Unexpected application failures | High | Low | Implement structured logging, health monitoring, and automated alerting. |

## 15.3 Technical Risks

Several technical risks are associated with AI-powered chatbot systems.

## 15.4 Operational Risks

Operational risks relate to deployment, infrastructure, and ongoing maintenance.

## 15.5 Business Risks

Business-related risks should also be considered.

## 15.6 Risk Review Process

Risks should be reviewed periodically during development and after production deployment.

The review process involves the following sequential checks:

1. **Metrics Analysis** Review API costs, token usage, and latency.
2. **Feedback Review** Analyze thumbs up/down and explicit user feedback.
3. **Query Audit** Identify failed, unanswered, or low-confidence queries.
4. **Quality Check** Assess the accuracy and hallucination rate of AI responses.
5. **Knowledge Audit** Expand knowledge base coverage to address gaps.



Continuous review allows the chatbot to evolve while maintaining reliability and accuracy.

## 15.7 Key Design Decisions

| Decision | Justification |
|---|---|
| **Early risk identification** | Reduces implementation and operational uncertainty. |
| **RAG architecture** | Minimizes hallucinations by grounding responses in approved content. |
| **Event-driven synchronization** | Ensures knowledge remains current without unnecessary processing. |
| **Operational monitoring** | Enables early detection of technical issues. |
| **Provider-independent architecture** | Reduces dependency on a single AI provider and improves future flexibility. |

# Chapter 16 – Future Roadmap

## 16.1 Overview

The Theta Chat Bot has been designed using a modular and extensible architecture that supports future enhancements with minimal changes to the existing system. While the initial implementation focuses on delivering accurate website-based responses using a Retrieval-Augmented Generation (RAG) architecture, the proposed design enables the chatbot to evolve incrementally as business requirements grow.

The roadmap below outlines the planned evolution of the chatbot while maintaining scalability, maintainability, and deployment flexibility.

## 16.2 Phase 1 – Initial Production Release (MVP)

The first phase focuses on delivering a production-ready chatbot integrated with the Theta Technolabs website.

| Component | Feature Deliverable |
|---|---|
| **Retrieval Engine** | Website-based knowledge retrieval utilizing RAG architecture. |
| **LLM Integration** | Google Gemini API integration (with OpenAI as a tested alternative). |
| **Embeddings** | Sentence Transformers for generating local vector embeddings. |
| **Backend** | FastAPI application with Docker-based deployment. |
| **Database** | PostgreSQL integration with pgvector for vector search. |
| **Frontend** | Webflow chatbot integration with session-based conversation memory. |
| **User Experience** | Source-aware responses and confidence-based fallback handling. |

## 16.3 Phase 2 – Monitoring & Analytics

Following the initial deployment, the focus shifts toward improving operational visibility, monitoring, and performance analysis.

| Focus Area | Feature Deliverable |
|---|---|
| **Administrative Dashboard** | Centralized interface for overall system visibility and management. |
| **Logging & Tracking** | Comprehensive request, response, and conversation history logging. |
| **System Monitoring** | Performance monitoring and continuous usage analytics tracking. |
| **Knowledge Auditing** | Knowledge synchronization history and automated audit trails. |
| **AI Insights** | AI-powered conversation summaries and direct user feedback collection. |
| **Security Controls** | Implementation of CAPTCHA and API rate-limiting. |

**Objective:** Improve operational monitoring and provide actionable insights into chatbot usage, conversation trends, and overall system performance.

ROADMAP

## 16.4 Phase 3 – User Experience Enhancements

This phase focuses on enhancing the overall user experience and making interactions more engaging and accessible.

| Enhancement | Feature Deliverable |
|---|---|
| **Interaction Modes** | Voice-based interaction and file upload support. |
| **Rich UI Components** | Rich response cards and dynamically suggested follow-up questions. |
| **Localization** | Multi-language support for diverse and global user bases. |
| **Advanced Memory** | Enhanced conversation memory mapping for long-running contexts. |
| **Accessibility** | Continued and targeted accessibility improvements for all users. |

## 16.5 Long-Term Vision

The long-term vision is to continuously enhance the Theta Chat Bot by improving response quality, user experience, and operational efficiency. Future enhancements will focus on increasing system performance, simplifying maintenance, and supporting evolving business requirements without requiring significant architectural changes.

| Strategic Area | Roadmap Deliverable |
|---|---|
| **Enterprise Scaling** | Multi-website support and continuous backend performance optimization. |
| **Advanced Retrieval** | Improved RAG techniques and enhanced semantic search accuracy. |
| **Deep Analytics** | Advanced conversation analytics and fully automated reporting. |
| **Response Quality** | Improved algorithmic response quality evaluation processes. |
| **Data Automation** | Fully automated knowledge synchronization pipelines. |

This design provides the flexibility to incorporate these enhancements incrementally while maintaining system stability.

ROADMAP

## 16.6 Roadmap Summary

| Phase | Primary Objective |
|---|---|
| **Phase 1** | Deploy a production-ready RAG chatbot and establish the core AI platform |
| **Phase 2** | Introduce monitoring, dashboards, and operational analytics |
| **Phase 3** | Enhance user experience and accessibility |
| **Long-Term Vision** | Expand the chatbot through continuous improvements and support for future business requirements |

The phased roadmap enables incremental development while minimizing implementation risk. Each phase builds upon the existing foundation, allowing the Theta Chat Bot to evolve into a scalable, intelligent customer support solution without requiring a complete system redesign.

# Appendix A – LLM Cost Comparison

## A.1 Overview

This appendix provides a comprehensive financial and operational cost breakdown for Large Language Model (LLM) providers and local embedding models evaluated for the Theta Chat Bot.

## A.2 LLM Pricing Reference

The table below details current market pricing (USD per 1 Million tokens) for the primary and fallback LLM candidate models.

| Model | Provider | Input Cost (/1M) | Output Cost (/1M) | Suitability |
|---|---|---|---|---|
| **Gemini 3.1 Flash Lite** | Google | $0.25 (₹24.08) | $1.50 (₹144.50) | Default choice for high-speed RAG Q&A |
| **GPT-5.4 mini** | OpenAI | $0.75 (₹72.25) | $4.50 (₹433.49) | Secondary comparison & fallback model |
| **Gemini 3.5 Flash** | Google | $1.50 (₹144.50) | $9.00 (₹866.97) | Advanced reasoning & multi-turn context |
| **GPT-5.6 Luna** | OpenAI | $1.00 (₹96.33) | $6.00 (₹577.98) | Enterprise fallback model |



## A.3 Per-Conversation Cost Calculation

Assuming an average user interaction consists of 2,000 input tokens (retrieved website context + prompt + question) and generates a 500-token response:

- **Input Cost:** 2,000 tokens × $0.25 / 1,000,000 = **$0.0005**
- **Output Cost:** 500 tokens × $1.50 / 1,000,000 = **$0.00075**
- **Total Cost per Conversation: ~$0.00125 USD (₹0.12 INR)**

> **ROI IMPACT**
>
> At this rate, **1,000 complete customer conversations cost approximately $1.25 USD (₹120.41 INR) in AI API fees**, ensuring extremely low operational expenses.

## A.4 Cost Reduction Strategies

| Strategy | Impact on Operational Expenses |
|---|---|
| **Local Embedding Models** | 100% savings on embedding API fees. Sentence Transformers runs locally on internal servers with zero per-token costs. |
| **Semantic Caching** | Caching answers for common questions (e.g., business hours, location, core services) reduces LLM API calls by 30–40%. |
| **Self-Hosted Vector Store** | Running Qdrant or PostgreSQL + pgvector in Docker containers avoids monthly third-party cloud vector DB subscriptions. |
| **Prompt Compression & Chunk Pruning** | Filtering out low-relevance chunks ensures context stays under 2,000 tokens, directly lowering input token billing. |

# Requirement Gathering Questions

## B.1 Project Scope

**Q1: Which website(s) should the chatbot support (e.g., only thetatechnolabs.in or multiple domains such as .com and .ca)?**

## B.2 Website Content & Knowledge Base

**Q1: How frequently is the website content updated (e.g., service pages, portfolios, case studies)?**

**Q2: How often are new blogs or articles published?**

**Q3: Will someone notify the team about content updates, or can we automate the knowledge base update process using the existing website CMS?**

## B.3 Existing Chatbot

**Q1: The existing website already has a manual inquiry chat widget on the right side of the website. Are conversation details such as user queries, responses, timestamps, and other interaction data currently being stored? If yes, where is this data stored, and can I access it?**

## B.4 Target Audience

**Q1: Who is the primary audience for the chatbot (e.g., potential clients, existing clients, job seekers, or general website visitors)?**

## B.5 Chatbot Placement

**Q1: Should the chatbot be available on every page of the website or only on selected pages?**

## B.6 Infrastructure

**Q1: Does the existing server have sufficient CPU, RAM, and storage to host the chatbot and its supporting services?**

## B.7 Administration

**Q1: Is an admin dashboard required for managing the knowledge base, chatbot configuration, or analytics?**

## B.8 AI Usage & Budget

**Q1: Is there a monthly budget limit for AI API usage?**